Constraint programming

Constraint programming (CP)[1] is a paradigm for solving combinatorial problems that draws on a wide range of techniques from artificial intelligence, computer science, and operations research. In constraint programming, users declaratively state the constraints on the feasible solutions for a set of decision variables. Constraints differ from the common primitives of imperative programming languages in that they do not specify a step or sequence of steps to execute, but rather the properties of a solution to be found. In addition to constraints, users also need to specify a method to solve these constraints. This typically draws upon standard methods like chronological backtracking and constraint propagation, but may use customized code like a problem-specific branching heuristic.

Constraint programming takes its root from and can be expressed in the form of constraint logic programming, which embeds constraints into a logic program. This variant of logic programming is due to Jaffar and Lassez,[2] who extended in 1987 a specific class of constraints that were introduced in Prolog II. The first implementations of constraint logic programming were Prolog III, CLP(R), and CHIP.

Instead of logic programming, constraints can be mixed with functional programming, term rewriting, and imperative languages. Programming languages with built-in support for constraints include Oz (functional programming) and Kaleidoscope (imperative programming). Mostly, constraints are implemented in imperative languages via *constraint solving toolkits*, which are separate libraries for an existing imperative language.

Constraint logic programming

Constraint programming is an embedding of constraints in a host language. The first host languages used were logic programming languages, so the field was initially called *constraint logic programming*. The two paradigms share many important features, like logical variables and backtracking. Today most Prolog implementations include one or more libraries for constraint logic programming.

The difference between the two is largely in their styles and approaches to modeling the world. Some problems are more natural (and thus, simpler) to write as logic programs, while some are more natural to write as constraint programs.

The constraint programming approach is to search for a state of the world in which a large number of constraints are satisfied at the same time. A problem is typically stated as a state of the world containing a number of unknown variables. The constraint program searches for values for all the variables.

Temporal concurrent constraint programming (TCC) and non-deterministic temporal concurrent constraint programming (MJV) are variants of constraint programming that can deal with time.

Constraint satisfaction problem

A constraint is a relation between multiple variables that limits the values these variables can take simultaneously.

Definition—A constraint satisfaction problem on finite domains (or CSP) is defined by a triplet $(\mathcal{X}, \mathcal{D}, \mathcal{C})$ where:

- $\mathcal{X} = \{x_1, \ldots, x_n\}$ is the set of variables of the problem;
- $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_n\}$ is the set of domains of the variables, i.e., for all $k \in [1; n]$ we have $x_k \in \mathcal{D}_k$;
- $\mathcal{C} = \{C_1, \ldots, C_m\}$ is a set of constraints. A constraint $C_i = (\mathcal{X}_i, \mathcal{R}_i)$ is defined by a set $\mathcal{X}_i = \{x_{i_1}, \ldots, x_{i_k}\}$ of variables and a relation $\mathcal{R}_i \subseteq \mathcal{D}_{i_1} \times \cdots \times \mathcal{D}_{i_k}$ that defines the set of values allowed simultaneously for the variables of $\mathcal{X}_i$.

Three categories of constraints exist:

- extensional constraints: constraints are defined by enumerating the set of values that would satisfy them;
- arithmetic constraints: constraints are defined by an arithmetic expression, i.e., using $<, >, \leq, \geq, =, \neq, \ldots$;
- logical constraints: constraints are defined with an explicit semantics, i.e., *AllDifferent*, *AtMost*,...

Definition— An assignment (or model) $\mathcal{A}$ of a CSP $P = (\mathcal{X}, \mathcal{D}, \mathcal{C})$ is defined by the couple $\mathcal{A} = (\mathcal{X}_\mathcal{A}, \mathcal{V}_\mathcal{A})$ where:

- $\mathcal{X}_\mathcal{A} \subseteq \mathcal{X}$ is a subset of variable;
- $\mathcal{V}_\mathcal{A} = \{v_{\mathcal{A}_1}, \ldots, v_{\mathcal{A}_k}\} \in \{\mathcal{D}_{\mathcal{A}_1}, \ldots, \mathcal{D}_{\mathcal{A}_k}\}$ is the tuple of the values taken by the assigned variables.

Assignment is the association of a variable to a value from its domain. A partial assignment is when a subset of the variables of the problem has been assigned. A total assignment is when all the variables of the problem have been assigned.

Property—Given $\mathcal{A} = (\mathcal{X}_\mathcal{A}, \mathcal{V}_\mathcal{A})$ an assignment (partial or total) of a CSP $P = (\mathcal{X}, \mathcal{D}, \mathcal{C})$, and $C_i = (\mathcal{X}_i, \mathcal{R}_i)$ a constraint of P such as $\mathcal{X}_i \subseteq \mathcal{X}_\mathcal{A}$, the assignment $\mathcal{A}$ satisfies the constraint C_i if and only if all the values $\mathcal{V}_{\mathcal{A}_i} = \{v_i \in \mathcal{V}_\mathcal{A} \textbf{ such that } x_i \in \mathcal{X}_i\}$ of the variables of the constraint C_i belongs to $\mathcal{R}_i$.

Definition—A solution of a CSP is a total assignment that satisfies all the constraints of

the problem.

During the search of the solutions of a CSP, a user can wish for:

- finding a solution (satisfying all the constraints);
- finding all the solutions of the problem;
- proving the unsatisfiability of the problem.

Constraint optimization problem

A constraint optimization problem (COP) is a constraint satisfaction problem associated to an objective function.

An *optimal solution* to a minimization (maximization) COP is a solution that minimizes (maximizes) the value of the *objective function*.

During the search of the solutions of a COP, a user can wish for:

- finding a solution (satisfying all the constraints);
- finding the best solution with respect to the objective;
- proving the optimality of the best found solution;
- proving the unsatisfiability of the problem.

Perturbation vs refinement models

Languages for constraint-based programming follow one of two approaches:[3]

- Refinement model: variables in the problem are initially unassigned, and each variable is assumed to be able to contain any value included in its range or domain. As computation progresses, values in the domain of a variable are pruned if they are shown to be incompatible with the possible values of other variables, until a single value is found for each variable.
- Perturbation model: variables in the problem are assigned a single initial value. At different times one or more variables receive perturbations (changes to their old value), and the system propagates the change trying to assign new values to other variables that are consistent with the perturbation.

Constraint propagation in constraint satisfaction problems is a typical example of a refinement model, and formula evaluation in spreadsheets are a typical example of a perturbation model.

The refinement model is more general, as it does not restrict variables to have a single value, it can lead to several solutions to the same problem. However, the perturbation model is more intuitive for programmers using mixed imperative constraint object-oriented languages.[4]

Domains

The constraints used in constraint programming are typically over some specific domains. Some popular domains for constraint programming are:

- Boolean domains, where only true/false constraints apply (SAT problem)
- integer domains, rational domains
- interval domains, in particular for scheduling problems[5]
- linear domains, where only linear functions are described and analyzed (although approaches to non-linear problems do exist)
- finite domains, where constraints are defined over finite sets
- mixed domains, involving two or more of the above

Finite domains is one of the most successful domains of constraint programming. In some areas (like operations research) constraint programming is often identified with constraint programming over finite domains.

Constraint propagation

Local consistency conditions are properties of constraint satisfaction problems related to the consistency of subsets of variables or constraints. They can be used to reduce the search space and make the problem easier to solve. Various kinds of local consistency conditions are leveraged, including **node consistency**, **arc consistency**, and **path consistency**.

Every local consistency condition can be enforced by a transformation that changes the problem without changing its solutions. Such a transformation is called **constraint propagation**.[6] Constraint propagation works by reducing domains of variables, strengthening constraints, or creating new ones. This leads to a reduction of the search space, making the problem easier to solve by some algorithms. Constraint propagation can also be used as an unsatisfiability checker, incomplete in general but complete in some particular cases.

Constraint solving

There are three main algorithmic techniques for solving constraint satisfaction problems: backtracking search, local search, and dynamic programming.[1]

Backtracking search

Backtracking search is a general algorithm for finding all (or some) solutions to some computational problems, notably constraint satisfaction problems, that incrementally builds candidates to the solutions, and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly be completed to a valid solution.

Local Search

Local search is an incomplete method for finding a solution to a problem. It is based on iteratively improving an assignment of the variables until all constraints are satisfied. In particular, local search algorithms typically modify the value of a variable in an assignment at each step. The new assignment is close to the previous one in the space of assignment, hence the name *local search*.

Dynamic programming

Dynamic programming is both a mathematical optimization method and a computer programming method. It refers to simplifying a complicated problem by breaking it down into simpler sub-problems in a recursive manner. While some decision problems cannot be taken apart this way, decisions that span several points in time do often break apart recursively. Likewise, in computer science, if a problem can be solved optimally by breaking it into sub-problems and then recursively finding the optimal solutions to the sub-problems, then it is said to have optimal substructure.

Example

The syntax for expressing constraints over finite domains depends on the host language. The following is a Prolog program that solves the classical alphametic puzzle SEND+MORE=MONEY in constraint logic programming:

```
% This code works in both YAP and SWI-Prolog using the environment-supplied
% CLPFD constraint solver library.  It may require minor modifications to work
% in other Prolog environments or using other constraint solvers.
:- use_module(library(clpfd)).
sendmore(Digits) :-
   Digits = [S,E,N,D,M,O,R,Y],            % Create variables
   Digits ins 0..9,                       % Associate domains to variables
   S #\= 0,                               % Constraint: S must be different from 0
   M #\= 0,
   all_different(Digits),                 % all the elements must take different values
                1000*S + 100*E + 10*N + D  % Other constraints
              + 1000*M + 100*O + 10*R + E
   #= 10000*M + 1000*O + 100*N + 10*E + Y,
   label(Digits).                         % Start the search
```

The interpreter creates a variable for each letter in the puzzle. The operator `ins` is used to specify the domains of these variables, so that they range over the set of values {0,1,2,3, ..., 9}. The constraints `S#\=0` and `M#\=0` means that these two variables cannot take the value zero. When the interpreter evaluates these constraints, it reduces the domains of these two variables by removing the value 0 from them. Then, the constraint `all_different(Digits)` is considered; it does not reduce any domain, so it is simply stored. The last constraint specifies that the digits assigned to the letters must be such that "SEND+MORE=MONEY" holds when each letter is replaced by its corresponding digit. From this constraint, the solver infers that M=1. All stored constraints involving variable M are awakened: in this case, constraint propagation on the `all_different` constraint removes value 1 from the domain of all the remaining variables. Constraint propagation may solve the problem by reducing all domains to a single value, it may prove that the problem has no solution by reducing a domain to the empty set, but may also terminate without proving satisfiability or unsatisfiability. The **label** literals are used to actually perform search for a solution.

See also

- Combinatorial optimization
- Concurrent constraint logic programming
- Constraint logic programming
- Heuristic algorithms
- List of constraint programming languages
- Mathematical optimization
- Nurse scheduling problem
- Regular constraint
- Satisfiability modulo theories
- Traveling tournament problem

References

1. Rossi, Francesca; Beek, Peter van; Walsh, Toby (2006). *Handbook of Constraint Programming* (https://books.google.com/books?id=Kjap9ZWcKOoC&pg=PP1). Elsevier. ISBN 978-0-08-046380-3.
2. Jaffar, Joxan; Lassez, J-L. (1987). "Constraint logic programming" (https://dl.acm.org/citation.cfm?id=41635). *POPL87: Fourteenth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery. pp. 111–9. doi:10.1145/41625.41635 (https://doi.org/10.1145%2F41625.41635). ISBN 978-0-89791-215-0.
3. Borning, A.; Freeman-Benson, B.; Wilson, M. (1993). "Constraint Hierarchies" (https://link.springer.com/chapter/10.1007/978-3-642-85983-0_4). In Mayoh, B.; Tyugu, E.; Penjam, J. (eds.). *Constraint Programming*. NATO ASI F. Vol. 131. Springer. p. 76. doi:10.1007/978-3-642-85983-0_4 (https://doi.org/10.1007%2F978-3-642-85983-0_4). ISBN 978-3-642-85983-0.
4. Lopez, G.; Freeman-Benson, B.; Borning, A. (1993). "Kaleidoscope: A constraint imperative programming language" (https://dada.cs.washington.edu/research/tr/1993/09/UW-CSE-93-09-04.pdf) (PDF). In Mayoh, B.; Tyugu, E.; Penjam, J. (eds.). *Constraint Programming*. NATO ASI F. Vol. 131. Springer. pp. 313–329. doi:10.1007/978-3-642-85983-0_12 (https://doi.org/10.1007%2F978-3-642-85983-0_12). ISBN 978-3-642-85983-0.
5. Baptiste, Philippe; Pape, Claude Le; Nuijten, Wim (2012). *Constraint-Based Scheduling: Applying Constraint Programming to Scheduling Problems* (https://books.google.com/books?id=qUzhBwAAQBAJ&pg=PR5). Springer. ISBN 978-1-4615-1479-4.
6. Bessiere, Christian (2006). "Constraint Propagation". *Handbook of Constraint Programming*. Foundations of Artificial Intelligence. Vol. 2. Elsevier. pp. 29–83. CiteSeerX 10.1.1.398.4070 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.398.4070). doi:10.1016/s1574-6526(06)80007-6 (https://doi.org/10.1016%2Fs1574-6526%2806%2980007-6). ISBN 9780444527264.

External links

- International Conference on Principles and Practice of Constraint Programming (https://www.a4cp.org/events/cp-conference-series). Association for Constraint Programming (https://www.a4cp.org/).

- Barták, Roman (1998). "On-line Guide to Constraint Programming" (http://kti.ms.mff.cuni.cz/~bartak/constraints/). Charles University, Prague.
- Deprecated link at archive.today (archived December 5, 2012), an Oz-based free software (X Window System style)
- Deprecated link at archive.today (archived January 7, 2013)
- Shirriff, Ken (October 2025). "Solving the NYTimes Pips puzzle with a constraint solver" (http://www.righto.com/2025/10/solve-nyt-pips-with-constraints.html).
- Scassellati, Brian (2019). "Constraint Satisfaction Problems" (https://zoo.cs.yale.edu/classes/cs470/lectures/s2019/07-CSP.pdf) (PDF). *CPSC 470 – Artificial Intelligence*. Yale University.
